# ОТЧЁТ

**по лабораторной работе №15**

**«Веб-приложение на Flask»**

Вариант 20. Task Manager

## 1. Цель работы

Научиться строить серверное веб-приложение на Flask: маршруты, шаблоны, формы и изменение состояния задачи через браузер.

## 2. Постановка индивидуального задания

Вариант 20. Сущности User, Project и Task. Нужно управлять проектами и задачами: видеть список, добавлять проект и задачу, назначать исполнителя и менять статус.

## 3. Краткие теоретические сведения

Flask сопоставляет адрес функции-представлению. Шаблон Jinja подставляет данные в HTML. Форма отправляет POST, представление меняет коллекцию и отвечает переадресацией на список, чтобы обновление страницы не повторяло отправку.

Статический файл стилей отдаётся отдельно от шаблона. Представление не содержит разметку таблицы: таблица лежит в templates/index.html.

## 4. Описание программы

В памяти хранятся пользователи Иванов и Петрова, проекты «Сайт провайдера» и «Биллинг» и две исходные задачи. Форма новой задачи выбирает проект, исполнителя и приоритет. Форма проекта выбирает владельца. У каждой строки есть выбор статуса и кнопка сохранения.

Во время проверки через запущенный сервер добавлена задача «Проверить канал» с высоким приоритетом и исполнителем Петровой. Снимок страницы показывает три строки списка.

## 5. Исходный код программы

Ниже приведён исходный код разработанных файлов.

### Листинг — app.py

```
"""
Лабораторная работа №15. Вариант 20.
Веб-приложение учёта проектов и задач на Flask.
"""
```

```
from flask import Flask, redirect, render_template, request, url_for

app = Flask(__name__)

USERS = [
    {"id": 1, "name": "Иванов"},
    {"id": 2, "name": "Петрова"},
]
PROJECTS = [
    {"id": 1, "title": "Сайт провайдера", "owner_id": 1},
    {"id": 2, "title": "Биллинг", "owner_id": 2},
]
TASKS = [
    {"id": 1, "title": "Макет главной", "project_id": 1, "assignee_id": 2, "status": "новая", "priority":
"высокий"},
    {"id": 2, "title": "Оплата тарифа", "project_id": 2, "assignee_id": 1, "status": "в работе",
"priority": "средний"},
]
NEXT = {"task": 3, "project": 3}



def user_name(user_id):
    for user in USERS:
        if user["id"] == user_id:
            return user["name"]
    return "—"



def project_title(project_id):
    for project in PROJECTS:
        if project["id"] == project_id:
            return project["title"]
    return "—"



@app.get("/")
def index():
    rows = []
    for task in TASKS:
        rows.append({
            **task,
            "project": project_title(task["project_id"]),
            "assignee": user_name(task["assignee_id"]),
        })
    return render_template("index.html", tasks=rows, projects=PROJECTS, users=USERS)



@app.post("/projects")
def add_project():
    title = request.form.get("title", "").strip()
    owner_id = int(request.form.get("owner_id", 1))
    if title:
        PROJECTS.append({"id": NEXT["project"], "title": title, "owner_id": owner_id})
        NEXT["project"] += 1
    return redirect(url_for("index"))



@app.post("/tasks")
def add_task():
    title = request.form.get("title", "").strip()
    if not title:
        return redirect(url_for("index"))
    TASKS.append({
        "id": NEXT["task"],
        "title": title,
        "project_id": int(request.form["project_id"]),
        "assignee_id": int(request.form["assignee_id"]),
        "status": "новая",
        "priority": request.form.get("priority", "средний"),
    })
    NEXT["task"] += 1
```

```
    return redirect(url_for("index"))


@app.post("/tasks/<int:task_id>/status")
def change_status(task_id):
    for task in TASKS:
        if task["id"] == task_id:
            task["status"] = request.form.get("status", task["status"])
    return redirect(url_for("index"))


if __name__ == "__main__":
    app.run(host="127.0.0.1", port=5015, debug=False)
```

# Листинг — static\style.css

```
body { font-family: "Segoe UI", sans-serif; margin: 0; background: #f4f6f8; color: #1d2430; }
header { background: #16324f; color: white; padding: 20px 32px; }
header h1 { margin: 0 0 6px; }
main { padding: 24px 32px; }
table { width: 100%; border-collapse: collapse; background: white; }
th, td { border-bottom: 1px solid #e3e8ee; text-align: left; padding: 10px; }
th { background: #eef3f8; }
.forms { display: flex; gap: 24px; margin-top: 24px; }
form { background: white; padding: 16px; min-width: 280px; display: flex; flex-direction: column; gap:
8px; }
input, select, button { padding: 8px; font: inherit; }
button { background: #16324f; color: white; border: 0; cursor: pointer; }
```

# Листинг — templates\index.html

```
<!DOCTYPE html>
<html lang="ru">
<head>
  <meta charset="utf-8">
  <title>Task Manager</title>
  <link rel="stylesheet" href="{{ url_for('static', filename='style.css') }}">
</head>
<body>
  <header>
    <h1>Task Manager</h1>
    <p>Проекты, задачи и исполнители</p>
  </header>
  <main>
    <section>
      <h2>Задачи</h2>
      <table>

<tr><th>ID</th><th>Задача</th><th>Проект</th><th>Исполнитель</th><th>Приоритет</th><th>Статус</th></tr>
        {% for task in tasks %}
        <tr>
          <td>{{ task.id }}</td>
          <td>{{ task.title }}</td>
          <td>{{ task.project }}</td>
          <td>{{ task.assignee }}</td>
          <td>{{ task.priority }}</td>
          <td>
            <form method="post" action="{{ url_for('change_status', task_id=task.id) }}">
              <select name="status">
                {% for status in ['новая', 'в работе', 'готово'] %}
                <option {% if task.status == status %}selected{% endif %}>{{ status }}</option>
                {% endfor %}
              </select>
              <button type="submit">Сохранить</button>
            </form>
          </td>
        </tr>
        {% endfor %}
      </table>
    </section>
    <section class="forms">
      <form method="post" action="{{ url_for('add_task') }}">
        <h2>Новая задача</h2>
        <input name="title" placeholder="Название" required>
        <select name="project_id">
          {% for project in projects %}<option value="{{ project.id }}">{{ project.title }}</option>{%
endfor %}
```

```
        </select>
        <select name="assignee_id">
          {% for user in users %}<option value="{{ user.id }}">{{ user.name }}</option>{% endfor %}
        </select>
        <select name="priority">
          <option>низкий</option><option selected>средний</option><option>высокий</option>
        </select>
        <button type="submit">Добавить задачу</button>
      </form>
      <form method="post" action="{{ url_for('add_project') }}">
        <h2>Новый проект</h2>
        <input name="title" placeholder="Название проекта" required>
        <select name="owner_id">
          {% for user in users %}<option value="{{ user.id }}">{{ user.name }}</option>{% endfor %}
        </select>
        <button type="submit">Добавить проект</button>
      </form>
    </section>
  </main>
</body>
</html>
```

## 6. Примеры работы программы

Снимок сделан с запущенного приложения по адресу http://127.0.0.1:5015/ после добавления задачи.

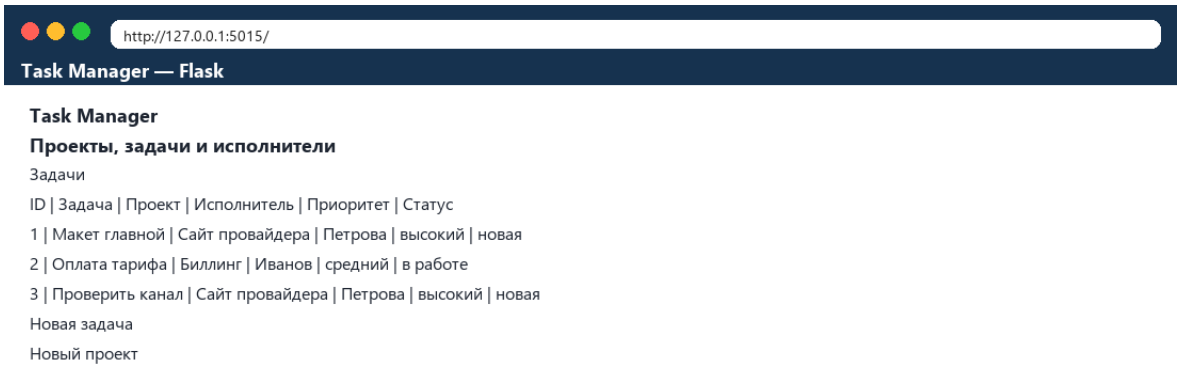



Рисунок 1 – Список задач Flask после добавления «Проверить канал»

## 7. Ответы на контрольные вопросы

### 1. Что такое представление во Flask?

Функция, которая обрабатывает адрес и возвращает HTML или перенаправление.

### 2. Зачем шаблон, если можно вернуть строку?

Разметка отделена от данных. Один шаблон показывает любой текущий список задач.

### 3. Почему после POST идёт redirect?

Иначе повторная загрузка страницы отправила бы форму ещё раз и создала бы дубликат.

### 4. Где хранятся данные этого варианта?

В списках процесса. Для учебной страницы этого достаточно; постоянная база показана в соседних работах.

**5. Как страница узнаёт имя исполнителя?**

Представление по идентификатору находит пользователя и проект и передаёт в шаблон уже готовые подписи.

## 8. Вывод

В лабораторной работе №15 на Flask собран Task Manager варианта 20. Страница показывает проекты и задачи, формы добавляют проект и задачу, статус меняется отправкой формы. Проверка на запущенном сервере добавила задачу и отобразила её в таблице.